Predefined Variables

The following variables are predefined in the Ruby interpreter. In these descriptions, the notation [r/o] indicates that the variables are read-only; an error will be raised if a program attempts to modify a read-only variable. After all, you probably don't want to change the meaning of true halfway through your program (except perhaps if you're a politician). Entries marked [thread] are thread local.

Many global variables look something like Snoopy swearing: $_, $!, $&, and so on. This is for "historical" reasons—most of these variable names come from Perl.

In the tables of variables and constants that follow, we list the variable name, the type of the referenced object, and a description.

Exception Information

$! → Exception	The exception object passed to raise. [thread]
$@ → Array	The stack backtrace generated by the last exception. [thread]

Input/Output Variables

$/ → String	The input record separator (newline by default). This is the value that routines such as gets use to determine record boundaries. If set to nil, gets will read the entire file.
$-0 → String	Synonym for $/.
$\ → String	The string appended to the output of every call to methods such as print and write. The default value is nil.
$, → String	The separator string output between the parameters to methods such as print and join. Defaults to nil, which adds no text.
$. → Fixnum	The number of the last line read from the current input file.
$; → String	The default separator pattern used by String#split. May be set using the -F command-line option.
$< → ARGF.class	Synonym for ARGF. See ARGF, on page 238.
$> → IO	The destination stream for print and printf. The default value is STDOUT.
$_ → String	The last line read by gets or readline. Many string-related functions in the Kernel module operate on $_ by default. The variable is local to the current scope. [thread]
$-F → String	Synonym for $;.
$stderr, $stdout, $stdin, → IO	The current standard error, standard output, and standard input streams.

Execution Environment Variables

$0 → String	The name of the top-level Ruby program being executed. Typically this will be the program's filename. On some operating systems, assigning to this variable will change the name of the process reported (for example) by the 'ps(1)' command.